MEMÓRIA · ENCONTRO 3 · LIGA PROTOFPGA

Folha de atividade

Nomes da dupla: _____ *Data:* __ / ___

Parte A — Atividade em aula (em duplas)

Trabalhem na mesma máquina, com o arquivo `aula3_vetor.asm` aberto no MARS. No menu Settings, deixem **marcado** *Addresses displayed in hexadecimal* e **desmarcado** *Values displayed in hexadecimal*. Cada parte tem tempo curto. Se travarem, chamem.

A1 — Observar (8 min)

1. Abram o `aula3_vetor.asm` e cliquem em Assemble. Na janela **Data Segment**, encontrem os cinco números do vetor. Em que endereço está o número 9? E o 5?

2. Na janela Text Segment, quantas linhas da coluna Basic correspondem a `la $s0, vetor`? Copiem o conteúdo dessas linhas.

3. Cliquem em Step duas vezes, até passar o `la`. Quanto vale `$s0` no painel de registradores? É o 7? O que é esse número?

A2 — Prever (8 min)

4. Antes de continuar, escrevam o que esperam:

`$t2 no final = _____` *Run I/O mostra:* _____

E qual endereço da memória vai mudar quando o `sw` executar? _____

5. Continuem com Step, um passo de cada vez. No passo do `sw`, olhem a janela Data Segment.

6. Erraram em algum ponto? Qual, e por quê?

A3 — Modificar (8 min)

7. Voltem à aba Edit e troquem o 9 do vetor por 90, na linha do `.word`. Antes de montar, escrevam o que deve aparecer no Run I/O: _____. Montem, executem e confiram. A coluna Code da janela Text Segment mudou?

8. Desfaçam a troca. Agora mudem `lw $t1, 8($s0)` para `lw $t1, 10($s0)`. Prevejam o que vai acontecer, montem e executem. O que o MARS disse? Por quê?

A4 — Provocar e explicar (6 min)

9. Desfaçam a troca. Na parte que imprime o resultado, troquem `lw $a0, 20($s0)` por `la $a0, soma`. Prevejam, montem e executem. Que número apareceu?

10. De onde veio esse número? Dica: procurem `soma` na tabela de rótulos da folha de consulta e convertam.

11. Escrevam UMA frase explicando a diferença entre `la $a0, soma` e `lw $a0, 20($s0)`. A frase precisa ser compreensível para alguém que não esteve nesta aula.

Parte B — Desafio da semana

Enunciado

Escreva no MARS um programa que:

1) leia três inteiros do teclado (serviço 5), guardando cada um numa posição de um vetor na memória com `sw`;

2) depois, busque os três de volta da memória com `lw` e some;

3) imprima `Soma =` seguido do resultado.

Declare o vetor com `nums: .word 0, 0, 0`. Teste com 4, 10 e −3 (os números são digitados no painel Run I/O). Depois, confira na janela Data Segment: em que endereços os três números ficaram guardados?

Espaço para o seu programa:

Parte C — Exercícios de revisão

Onze perguntas, para fazer em casa. As duas últimas pedem o MARS. Tentem responder tudo antes de abrir o gabarito — conferir cedo demais desperdiça o exercício.

1. Explique, em uma frase, a diferença entre um endereço e um valor.

2. Por que os endereços de palavras consecutivas pulam de 4 em 4?

3. Um vetor de palavras começa no endereço `0x10010000`. Qual é o endereço de `vetor[3]`? E o de `vetor[10]`? Responda em hexadecimal.

4. Na instrução `lw $t1, 12($s0)`, qual é o papel de cada uma das três partes — `$t1`, `12` e `$s0`? Qual delas contém um endereço?

5. Verdadeiro ou falso: depois de `sw $t2, 20($s0)`, o registrador `$t2` passa a valer o que estava na memória. Justifique.

6. `la` é uma instrução do processador? Em que ela se transforma, e por que são necessárias duas instruções?

7. Com `$s0 = 0x10010000`, por que `lw $t0, 6($s0)` causa erro? Que deslocamento você usaria para ler a palavra formada pelos bytes 4, 5, 6 e 7?

8. O número `0xAE0A0014` é uma instrução. Decomponha-o nos campos do formato I e escreva a instrução em assembly. Use a folha de consulta.

9. Para cada objetivo, diga o que vai em `$v0` e o que vai em `$a0`: (a) imprimir o número que está em `$t5`; (b) imprimir o texto que começa no rótulo `aviso`; (c) ler um número do teclado — e onde ele chega?; (d) encerrar o programa.

10. Sem usar o MARS, diga os valores finais de `$t1`, `$t2` e `$t3` e como fica a memória de `x` depois de executar:

```
        .data
x:      .word  10, 20, 30
        .text
main:   la   $t0, x
        lw   $t1, 4($t0)
        lw   $t2, 8($t0)
        add  $t3, $t1, $t2
        sw   $t3, 0($t0)
```

11. Escreva no MARS um programa que troque de lugar, na memória, o primeiro e o último elemento de `vetor: .word 7, 3, 9, 2, 5`. Confira na janela Data Segment que o vetor termina como 5, 3, 9, 2, 7. Quantos registradores temporários você precisou usar? Por que não dá para fazer com um só?

MEMÓRIA · ENCONTRO 3 · LIGA PROTOFPGA

Folha de consulta

Tudo o que a atividade e os exercícios pedem. Âmbar é endereço; verde é valor.

Mapa da memória (MARS)

Região	Começa em	Guarda
Pilha	`0x7FFFEFFC`	dados de funções — cresce para baixo (encontro 5)
Dados — `.data`	`0x10010000`	vetores, variáveis, textos
Código — `.text`	`0x00400000`	as instruções

Rótulos do aula3_vetor.asm

Rótulo	Endereço (hex)	Endereço (decimal)	Conteúdo
`vetor`	`0x10010000`	268 500 992	7, 3, 9, 2, 5
`soma`	`0x10010014`	268 501 012	0 (26 depois do sw)
`msg`	`0x10010018`	268 501 016	"Soma = " e um byte zero

Endereço de vetor[i] = base + 4 · i

i	0	1	2	3	4	5	6	7
4 · i	0	4	8	12	16	20	24	28
hex	0x00	0x04	0x08	0x0C	0x10	0x14	0x18	0x1C

Instruções de memória — formato I

op	rs	rt	imediato
`lw 100011 · sw 101011`	`base`	`lw: destino`	`deslocamento`
lw = 35 · sw = 43	endereço	sw: origem	16 bits, em bytes, com sinal

Escrita	Faz	Exemplo
`lw rt, desl(rs)`	rt ← Mem[rs + desl]	`lw $t1, 8($s0)`
`sw rt, desl(rs)`	Mem[rs + desl] ← rt	`sw $t2, 20($s0)`
`la reg, rótulo`	reg ← endereço do rótulo (pseudo: `lui` + `ori`)	`la $s0, vetor`

Chamadas de sistema

$v0	Serviço	$a0 / resultado
1	imprimir inteiro	$a0 = **valor**
4	imprimir texto	$a0 = **endereço** do texto
5	ler inteiro	resultado chega em $v0
10	encerrar	—
11	imprimir caractere	$a0 = código (10 = nova linha)

Registradores

Nome	Nº	Binário	Nome	Nº	Binário	Nome	Nº	Binário	Nome	Nº	Binário
$zero	0	00000	$t0	8	01000	$s0	16	10000	$t8	24	11000
$at	1	00001	$t1	9	01001	$s1	17	10001	$t9	25	11001
$v0	2	00010	$t2	10	01010	$s2	18	10010	$k0	26	11010
$v1	3	00011	$t3	11	01011	$s3	19	10011	$k1	27	11011
$a0	4	00100	$t4	12	01100	$s4	20	10100	$gp	28	11100
$a1	5	00101	$t5	13	01101	$s5	21	10101	$sp	29	11101
$a2	6	00110	$t6	14	01110	$s6	22	10110	$fp	30	11110
$a3	7	00111	$t7	15	01111	$s7	23	10111	$ra	31	11111

Hexadecimal ↔ binário

0 0000	**1** 0001	**2** 0010	**3** 0011	**4** 0100	**5** 0101	**6** 0110	**7** 0111
8 1000	**9** 1001	**A** 1010	**B** 1011	**C** 1100	**D** 1101	**E** 1110	**F** 1111